information with their inbuilt knowledge, so as to use the information later to better respond to their environment.

The Evolutionary Robotics Group at the University of Sussex evolved—not trained—a robot to identify a triangle in a room. The robot was not programmed with any knowledge of geometric shapes; it was evolved over several generations, and the evolution was stopped when it had learned to identify a triangle. And yes, robots have even evolved a language to communicate with their foster parents, the researchers in the labs they were born in.

Evolvable hardware

Hugo de Garis, one of the pioneers in the field of evolvable hardware (E-Hard), has a useful definition of the term. According to him, E-Hard is the marriage of reconfigurable hardware and evolutionary algorithms, often genetic algorithms *(See box 'Genetic Algorithms', page 25)*. Think of the hardware as living tissue. It needs to be programmable; that is, its behaviour—like that of living tissue—can be dictated by software of some sort. That's where FPGAs come in.

Living tissues have chromosomes that direct how the tissue will behave. Chromosomes come in strings of basic units, and their message is encoded in the way the basic units are strung together. In the case of E-Hard, the chromosomes used are genetic algorithms, encoded into bits. These cause the hardware to behave in a certain way. The codes that make the hardware perform better are allowed to have more offspring—the number of pieces of hardware they command—in the next generation of the algorithm. Evolution occurs by randomly mutating the chromosomes that define the hardware's behaviour, as it happens in nature.

Think of a robot that has two motors. If one of the motors fails, the robot will malfunction. In anthropomorphic terms, the robot wouldn't know what to do. However, if the hardware were evolvable, the robot would eventually repair the faulty motor, and begin to move properly.

Evolving robots: the Implications

Consider, from a designer's perspective, what evolving robots would mean. Instead of having to technically detail how a bot would go about exhibiting each bit of its desired behaviour, he would simply specify what he wanted from the bot.

Engineers would have an easier time because, with regular robots, the overall behaviour needs to be divided into modules, and each of these has to be described precisely. If the robot could evolve, a generic design combined with the designer's specifications would do, and the required 'global' behaviour might emerge. Robots could be mass produced and allowed to do their duties in special environments. The good ones would be used, and the bad ones simply thrown away!

How? Recall the Gödel machine. A robot equipped with a Gödel machine for a controller can modify its software; and with modifiable hardware, it can change its physical circuitry. Now consider such a robot that is given a goal. It would do some part of its job, inspect what went wrong and what went right, check its own code and hardware, and modify itself suitably... on and on, until it became perfect, or until it was destroyed—if it took too long.

Evolving robots know what they are. They know what they're supposed to do. They can change themselves. They're alive!

The bots have arrived

Sun Microsystems co-founder Bill Joy's now well-known article in Wired, titled 'Why the future doesn't need us', began with the subheading: "Our most powerful 21st-century technologies—robotics, genetic engineering, and nanotech—are threatening to make humans an endangered species." Joy went on to express his fears about the new technologies, saying, for example, that robots might be dangerous because they might replicate; that we may not survive our encounter with the new species, and so on.

Everyone has a point of view, and something to say about this—some say Joy is exaggerating, some share his concerns. But there's no doubting the fact that robotics has come of age; it's only a matter of time before we relegate more and more of our duties to the machines.

The 'new species'—humanoid, bot, part-human-part-bot, whatever—has already been born. ■

RAM MOHAN RAO
ram_mohan@thinkdigit.com

Training and Evolving Neural Networks

Neural networks consist of layers of processing units connected to each other. These connections have weights; think of the weight as the 'importance' of the connection, or how much of a role it plays in the entire network.

Inputs come in at the lowest layer, and the output is from the output layer. This is how neural nets learn, or are trained: the inputs are given, and an output is obtained. The output depends on the input, the connection weights, and on the topology of the network—that is, how many nodes and layers it has.

If the topology has already been decided upon, it is the connection weights that are changed according to the output. The output is checked, and its deviation from the ideal output dictates how the weights will be changed. After the weights have been changed, the output will be different the next time round. This new output is then used to change the weights again, and so on. The process is repeated until the output reaches an optimum.

The neural net can evolve in one of many ways, including using a genetic algorithm. Notice the similarity between the training of a neural net and the evolving of a genetic algorithm. It is easy to see how a genetic algorithm's evolution can be applied to a neural net. Each generation of a genetic algorithm produces new collections of strings. The neural net can change its topology and connection weights, based on the contents of the collection of strings. For example, the bits in the strings that the genetic algorithm has produced may translate directly into connection weights.

Jürgen Schmidhuber's homepage:
www.idsia.ch/~juergen/
The Evolutionary Robotics Homepage:
gral.ip.rm.cnr.it/evorobot/
Evolutionary and Adaptive Systems at the University of Sussex:
www.cogs.susx.ac.uk/easy/index.html
Bill Joy's article in Wired Magazine:
www.wired.com/news/gizmos/0,1452,50247,00.html